ACTIVITY-1: NETWORK TAB OF FIREFOX DEVELOPER TOOL
ACTIVITY-2: STORAGE TAB OF FIREFOX DEVELOPER TOOL
DOWNLOAD ALL SUBMISSIONS

Activity-1: Network Tab of Firefox developer tool

ATTEMPT DOWNLOAD SUBMISSION

Objective: Learn usage of network inspector of firefox developer tool

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "**Laksh:SecretPwd!"**

This is a demonstration of the network tab of the firefox developer tool. Look at the questions below and answer the questions by opening the relevant page and network tab for that page in Firefox Devtools (right click and Inspect element or Ctrl + Shift + I or F12 on Windows and Linux, or Cmd + Opt + I on macOS). You can use the dustbin icon to clear things, use the refresh circle (next to URL bar) to reload and disable cache to ensure nothing is being served locally.

1. How many successful http requests were made when the `login` page was loaded for the first time (do not include "favicon.ico")? Write the answer in `login_http_nos.txt`.

2. What was the endpoint which failed on `user` page (do not consider favicon.ico here also)? Write the answer in `failed_endpoint.txt`

3. What was the status code for above? Write the answer in `failed_status_code.txt`

4. Edit and resend the request of "failed-endpoint" where you have to edit the "User-Agent" header value to "Secret Agent 007" and you also need to add a new header with key "X-Secret" and value "Bond007". Write the flag you get in `flag.txt`

5. What was the status code for `logout` endpoint? Write the answer in `logout_status_code.txt`

Submission

Enter the answers in the relevant files and click evaluate to grade your solution. Post this, make a submission, select local directory and submit.

Reference:

https://firefox-source-docs.mozilla.org/devtools-user/network_monitor/index.html

ACTIVITY-1: NETWORK TAB OF FIREFOX DEVELOPER TOOL
ACTIVITY-2: STORAGE TAB OF FIREFOX DEVELOPER TOOL
DOWNLOAD ALL SUBMISSIONS

Activity-2: Storage Tab of Firefox developer tool

ATTEMPT DOWNLOAD SUBMISSION

Objective: Learn usage of storage tab of firefox developer tool

Problem Statement

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

This is a demonstration of the storage tab of the firefox developer tool. To solve this lab, use storage inspector to answer following questions

1. Clear all cookies. Load the `login` page. You will see only 1 cookie now. Write the Name of cookie in `first_cookie.txt

2. Now on the `login` page, clear the above cookie too, do not refresh and try to login. What is the status code when you try to login? (Recall network monitor). Write the status code in `status_code.txt`

3. Now load the `login` page again, use credentials to `login`. Notice that there is a new cookie `sessionid` present now, this is used to maintain your session with the backend. What is the value of `SameSite` attribute for this cookie? Write in `sessionid_SameSite.txt`.

4. Find the flag in cookies. Write the Value of flag in `flag1.txt`.

5. Create a new cookies named with key "secret-cookie" and value "getflag" and refresh the page. submit the new flag in "flag2.txt"

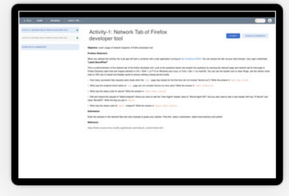
Submission

Enter the answers in the relevant files and click evaluate to grade your solution. Post this, make a submission, select local directory and submit.

Reference:

https://firefox-source-docs.mozilla.org/devtools-user/storage_inspector/index.html

Samesite: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie#samesitesamesite-value



ACTIVITY-1: INTRODUCTION TO ZAP
ACTIVITY-2: SCANNING WEB-APPLICATION USING ZAP
ACTIVITY-3: FUZZING VIA ZAP
DOWNLOAD ALL SUBMISSIONS

Activity-1: Introduction to ZAP

ATTEMPT

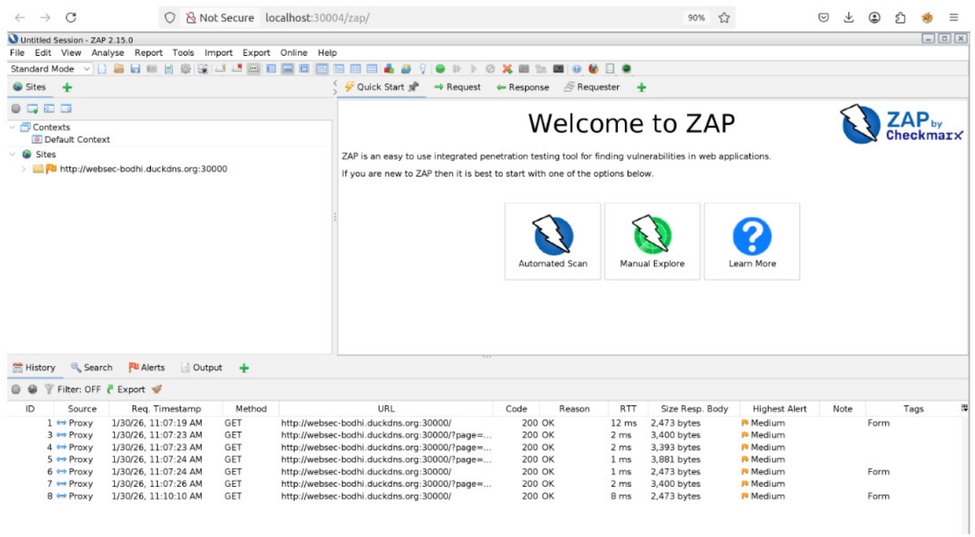
Objective: Setup ZAP and ensure it is working properly.

Problem Statement:

When you attempt this activity, the cLab app will start two services in a container. One is the ZAP service running at **http://localhost:30004/zap/** and another is a web application running at **http://websec-bodhi.duckdns.org:30000.** You can access them both via your web browser.

The goal is to setup FoxyProxy (that will direct browser traffic to ZAP for analysis) and also ZAP service to be able to see this traffic.

How to do this? See reference material below. There is no autograding for this exercise. If you see the traffic like in the figure shown below, your task is done.



Guidance:

- See https://docs.google.com/document/d/1ymE3-YQDYiWPAzvrKNeOoyB0mfOj9lsKYmsadCCKhH0/edit#heading=h.aczyuw2yex2w

ACTIVITY-1: INTRODUCTION TO ZAP
ACTIVITY-2: SCANNING WEB-APPLICATION USING ZAP
ACTIVITY-3: FUZZING VIA ZAP
DOWNLOAD ALL SUBMISSIONS

Activity-2: Scanning web-application using ZAP

ATTEMPT DOWNLOAD SUBMISSION

Objective: Use ZAP to perform vulnerability scan over both a normal website without authentication and one that requires authentication i.e. user login. Generate relevant reports of the scan for analysis.

Problem Statement:

When you attempt this activity, the cLab app will start three services in a container. One is the ZAP service running at **http://localhost:30004/zap/** , one is a web application running at **http://websec-bodhi.duckdns.org:30000** without authentication and one is the web application running at **http://websec-bodhi.duckdns.org:30001/** with authentication. You can access them all via your web browser.

Ensure Foxyproxy is set up correctly to direct all traffic of the web applications to ZAP. Note: we have done this as part of activity 1, though here we have two web apps. This means you may have to add two patterns.

As mentioned, your job is to do a vulnerability scan for both the web applications and generate a scan report and upload the same. Upload the report under report.html for part-1 (no authentication) and part-2 (with authentication). For part-2, you can use username:password as **Laksh:SecretPwd!** (! is part of the password)

Guidance:

See https://docs.google.com/document/d/1weHw4zHzpQf-F3opMPlodW5PWfYIGnNr9rhRt3t2oI0/edit#heading=h.1bhjazp36qhv

Submission:

Submit the generated report for http://websec-bodhi.duckdns.org:30000 in "/home/labDirectory/part1/report.html

Submit the generated report for http://websec-bodhi.duckdns.org:30001 in "/home/labDirectory/part2/report.html

And click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.

References:

https://www.zaproxy.org/getting-started/

https://www.triad.co.uk/news/owasp-zap/

https://medium.com/@lexuspaigelott/hands-on-lab-penetration-testing-with-owasp-zap-941df55b6505

https://medium.com/@secureica/authenticated-scan-using-owasp-zap-f0a71dafe41

ACTIVITY-1: INTRODUCTION TO ZAP
ACTIVITY-2: SCANNING WEB-APPLICATION USING ZAP
ACTIVITY-3: FUZZING VIA ZAP
DOWNLOAD ALL SUBMISSIONS

Activity-3: Fuzzing via ZAP

ATTEMPT DOWNLOAD SUBMISSION

Problem Statement:

Fuzzing is a testing technique used to find security vulnerabilities and bugs in software applications. It involves providing invalid, unexpected, or random data inputs to a computer program with the intention of causing it to crash, behave unexpectedly, or expose security vulnerabilities.

When you attempt this activity, the cLab app will start two services in a container. One is the ZAP service running at http://localhost:30004/zap/ , one is a web application running at http://websec-bodhi.duckdns.org:30000. You can access them all via your web browser.

You must have noticed by now that the web-application uses the "page" variable to navigate around the web-application. For Example: If you need to go to "student" page then it would be http://websec-bodhi.duckdns.org:30000?page=student

ZAP can be used to uncover hidden pages via fuzzing. Your task is to fuzz this variable to find the hidden web-page. For this you are also given a wordlist in the lab directory.

Guidance:

See

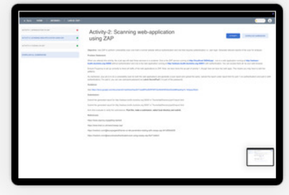
https://docs.google.com/document/d/1205wSsJm40Y5T2isv7VCJl9Wt0cnvgE3Gj5A43InEW8/edit?usp=sharing

Submission:

Once you find the hidden web-page, you will also get the flag (Format: flag{...}). Submit the flag in flag.txt file available in your lab directory. And click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.

Supporting Materials:

https://samedesilva.medium.com/how-to-use-fuzzing-feature-in-owasp-zap-2-9-0-48df6a89bef8



ACTIVITY-1: INFORMATION DISCLOSURE VIA COMMENTS
ACTIVITY-2: INFORMATION DISCLOSURE VIA ERROR/DEBUG MESSAGES
ACTIVITY-3: INFORMATION DISCLOSURE VIA BACKUP FILES
DOWNLOAD ALL SUBMISSIONS

Activity-1: Information Disclosure via comments

ATTEMPT DOWNLOAD SUBMISSION

Activity-1: Information Disclosure via comments

Objective: Obtain sensitive information left by developers in comments

Flag format: FLAG{...}

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "**Laksh:SecretPwd!"**

Your task is to find 5 flags hidden in the application. Think of these flags as some sensitive information accidentally leaked by the developers.

Guidance:

A few hints to help you out are as below:

- Firefox developer tools, specifically Page inspector, Style editor and Debugger should be useful. Be sure to use the search field.
- Look at all relevant HTML files, CSS files, javascript files as well as robots.txt

Submission:

- Submit all 5 flags you find via "flag.txt" available in "/home/labDirectory". (Order of flags does not matter)
- Submit all 5 exact URLs where you found these flags in "urls.txt" available in "/home/labDirectory". (Order of URLs does not matter)

Click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.

ACTIVITY-1: INFORMATION DISCLOSURE VIA COMMENTS
ACTIVITY-2: INFORMATION DISCLOSURE VIA ERROR/DEBUG MESSAGES
ACTIVITY-3: INFORMATION DISCLOSURE VIA BACKUP FILES
DOWNLOAD ALL SUBMISSIONS

Activity-2: Information Disclosure via error/debug messages

ATTEMPT DOWNLOAD SUBMISSION

Activity-2: Information Disclosure via error/debug messages

Objective: Obtain sensitive information left in error/debug messages

Flag format: FLAG{...}

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "**Laksh:SecretPwd!"**

Your task is to find 3 hidden flags. Think of these flags as some sensitive information accidentally leaked by the developers.

Guidance:

A few hints to help you out are as below:

- You have to somehow induce an error and make the website dump some debug messages. Determine the right URL and parameter needed to do this.
- You should be able to get 2 Flags directly from the debug message. Be sure to do intelligent search/browsing based on what function would have induced the error in the backend.
- To get the third flag, you need to extract some sensitive info from the dump and then use that info to interact with the website to get the flag.
- To be specific you will receive last flag when you login as "admin" user.

Submission:

- Submit all flags you find via "flag.txt" available in "/home/labDirectory". (Order of flags does not matter)
- Submit the answers of the questions asked in "answers.txt" file by replacing "<ANSWER-HERE>" for each question.

Click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.

ACTIVITY-1: INFORMATION DISCLOSURE VIA COMMENTS

ACTIVITY-2: INFORMATION DISCLOSURE VIA ERROR/DEBUG MESSAGES

ACTIVITY-3: INFORMATION DISCLOSURE VIA BACKUP FILES

DOWNLOAD ALL SUBMISSIONS

Activity-3: Information Disclosure via Backup files

ATTEMPT DOWNLOAD SUBMISSION

Activity-3: Information Disclosure via Backup files

Objective: Obtain sensitive information present in backup files

Flag format: FLAG{...}

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://websec-bodhi.duckdns.org:30000. You can access the site via your web browser. Use Login credentials: "**Laksh:SecretPwd!"**

This activity could make use of ZAP as well for fuzzing to speed up discovery. You can access ZAP at **http://localhost:30004/zap/.**

Your goal is to login as an "admin" user to get a flag. But to login, you need an admin password. Maybe it is available in some backup dump?

Guidance:

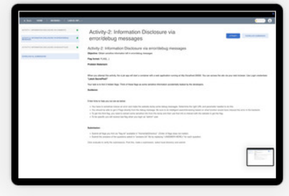
A few hints to help you out are as below:

- Where can you possibly find a list of backup folders?
- Once you discover such a list, note not all folders you discovered may be accessible still (web admin may have removed some folders later). But, once you find the list, you can use ZAP to enumerate through the folders to see what is accessible?
- When using ZAP, be sure to disable HUD, visit the web site at http://localhost:30000 and then enable foxyproxy.
- Construct a URL and visit it via browser to see if you can access the backup folder yourself first. This visit will result in a GET request in ZAP, and you can use that request to fuzz. Follow the same steps for fuzzing as before, except use Type as Strings and cut/paste the list under contents.
- Based on response code determine which folder(s) is worth exploring more
- Go through the files in the folder and see if one of them reveals the admin password

Submission

- Submit all flags you find via "flag.txt" available in "/home/labDirectory". (Order of flags does not matter)
- Submit the answers of the questions asked in "answers.txt" file by replacing "<ANSWER-HERE>" for each question.

Click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.



ACTIVITY 1: BASIC SSRF AGAINST LOCAL SERVER
ACTIVITY-3: SSRF AGAINST BLACKLIST BASED INPUT FILTER
ACTIVITY-4: SSRF AGAINST WHITELIST BASED INPUT FILTER
DOWNLOAD ALL SUBMISSIONS

Activity 1: Basic SSRF against local server

ATTEMPT DOWNLOAD SUBMISSION

Activity 1: Basic SSRF against local server

Objective: Access admin page present on local server and abuse its functionality

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

The website has an SSRF vulnerability. Find where the vulnerability is and abuse it to visit an admin panel hosted on the local server(http://localhost:30000/v1/admin). Now how do you know the URL of this admin panel? Assume you obtained it by guessing or through some prior information disclosure attack. In the admin panel, you will notice that you can add/delete users for the website. Extend the attack further and delete user "Neha"

For extra fun, try adding a new user as well.

Guidance:

A few hints to help you out are as below:

- First, directly visit the admin panel by typing this URL in the browser. You will notice that access is forbidden as it should.
- Login with provided credentials and navigate the website to see if there is any action that is vulnerable to SSRF. For this, while navigating, you also need to use the Page Inspector tool to see what requests are being made to which URLs/APIs (internal or external) due to the action.
- Once you identify the right endpoint that looks vulnerable to SSRF, replace the URL with the admin URL and see if you can access the page. You can do this editing via Page Inspector itself. Post editing be sure to redo the action/resend, so the request is made to the modified URL.
- If you are successful, you should now be able to see the admin page. Again explore this page via page inspector to see how you can delete a user.
- Do click on the delete button on the admin page. Again you will notice that this action is forbidden since you directly cannot perform such actions. You again need to leverage the trust of the local server to perform it for you via SSRF. Once you determine the right URL for delete, go back to the action that was vulnerable to SSRF and use this URL there.
- Perform the action and verify that the user is indeed deleted. You will get a flag once you successfully delete the user. Submit this flag.
- As an extra, based on these findings, try to add a new user. A bit more complicated, but boils down to constructing the right URL based on exploring the add new user page. Note, this gives you no extra flag and is optional to do.

Submission:

- Submit the URL you used to delete the user in the exploit.txt file.
- Submit the flag in flag.txt available in "/home/labDirectory".

Click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.

ACTIVITY 1: BASIC SSRF AGAINST LOCAL SERVER
ACTIVITY-3: SSRF AGAINST BLACKLIST BASED INPUT FILTER
ACTIVITY-4: SSRF AGAINST WHITELIST BASED INPUT FILTER
DOWNLOAD ALL SUBMISSIONS

Activity-3: SSRF against blacklist based input filter

ATTEMPT DOWNLOAD SUBMISSION

Activity-3: SSRF against blacklist based input filter

Objective: Access admin page present on local server and abuse its functionality

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

The website has an SSRF vulnerability. Find where the vulnerability is and abuse it to visit an admin panel hosted on the local server(http://localhost:30000/v1/admin). In the admin panel, you will notice that you can add/delete users for the website. Extend the attack further and delete user "Neha"

This is similar to the earlier activity, except that there are some defenses in the form of blacklisting in place. The blacklisted words are: ['127.0.0.1', '127.0.0.1:30000', '::1', 'localhost', 'localhost:30000', 'admin', 'delete', 'adduser']. You need to overcome the defenses here.

Guidance:

- Try to follow the same process as in activity 1 and also use the same URL that worked, to launch SSRF. You will now notice that it will not work, saying that the URL is blacklisted.
- You now have to overcome the defense both when accessing the admin panel i.e. http://localhost:30000/v1/admin as well as when deleting the user.
- Some kind of encoding is needed to overcome the defense!

Submission:

- Submit the URL you used to access the admin page as well as the URL you used to delete the user in the exploit.txt file.
- Submit the flag in flag.txt available in "/home/labDirectory".

Click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.

ACTIVITY 1: BASIC SSRF AGAINST LOCAL SERVER
ACTIVITY-3: SSRF AGAINST BLACKLIST BASED INPUT FILTER
ACTIVITY-4: SSRF AGAINST WHITELIST BASED INPUT FILTER
DOWNLOAD ALL SUBMISSIONS

Activity-4: SSRF against whitelist based input filter

ATTEMPT DOWNLOAD SUBMISSION

Activity-4: SSRF against whitelist based input filter

Objective: Access admin page present on local server and abuse its functionality

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

The website has an SSRF vulnerability. Find where the vulnerability is and abuse it to visit an admin panel hosted on the local server(http://localhost:30000/v1/admin). In the admin panel, you will notice that you can add/delete users for the website. Extend the attack further and delete user "Neha"

This is similar to the earlier activity, except that there are some defenses in the form of whitelisting in place (note there is no blacklisting here, only whitelisting). If you try to access the admin panel as in activity 1 or try some encoding to access as in activity 2, it will not succeed. It will say only a specific listed URL is allowed. You need to overcome that defense here.

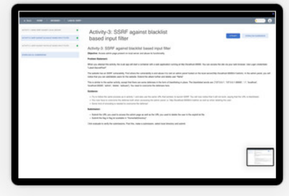
Guidance:

- Try both the URLs that worked to launch the SSRF attack in activity 1 and activity 2, they should not work here.
- Follow similar steps as in activity 1, except you have to overcome the defense both when accessing the admin panel i.e. http://localhost:30000/v1/admin as well as when deleting the user.
- Somehow the whitelisted URL has to appear in your attack url, but at the same time, you should also be able to access the admin panel.

Submission:

- Submit the URL you used to access the admin page as well as the URL you used to delete the user in the exploit.txt file.
- Submit the flag in flag.txt available in "/home/labDirectory".

Click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.



ACTIVITY-1: REMOTE CODE EXECUTION VIA SIMPLE ARBITRARY FILE UPLOAD

ACTIVITY-2: REMOTE CODE EXECUTION WITH BLACKLISTED EXTENSIONS BYPASS

ACTIVITY-3: REMOTE CODE EXECUTION WITH WHITELISTED FILE EXTENSION AND IMAGE CHECK BYPASS

DOWNLOAD ALL SUBMISSIONS

Activity-1: Remote Code Execution via Simple Arbitrary File Upload

ATTEMPT DOWNLOAD SUBMISSION

Activity-1: Remote Code Execution via Simple Arbitrary File Upload

Objective: Upload files containing commands to obtain sensitive information

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

When you click on your name, the website will take you to a page "http://localhost:30000/userDetails.php" which lets you change your user details, like email and profile photo.

Ideally the update profile picture feature should allow you to only upload image files (i.e. jpeg,jpg,png), but this has a file upload vulnerability.

Exploit this vulnerability to upload a malicious "PHP" script that prints the output of "id" command. If you do this correctly, you will receive a flag.

Extra Fun:

We have also provided a more advanced version of the reverse-shell code in the file "c99shell.php". Explore this too, it has many interesting features: you can list folder/files; run commands, create folders etc. **The c99shell.php is code taken from "https://github.com/phpwebshell/c99shell"**

Guidance:

- Craft the right php file and upload. While you can edit the shell.php in the cLab editor, for you to upload via browser, you need to locate this file from the host machine. This is possible, but convoluted. So, go ahead edit the shell.php file, but for easier upload, cut/paste the content to a local file (e.g. under Documents or Downloads) in the host machine and upload the local copy..
- If the uploaded file is correct, the autograder will give you the flag. Note, this may take a minute or more, so please wait and check.
- But the fun is in executing the script yourself and seeing the output of the id command. For this, you need to figure out where the file got uploaded. Use firefox page inspector to determine the file location.
- Visit that URL and click on the file. This should execute the file and show you the response of the execution.
- Post this, for more fun, be sure to also upload the c99shell.php and play around with it. Again cut/paste the content to a local file and upload from there.

Submission

- Submit the flag you find via "flag.txt" available in "/home/labDirectory".
- Submit the exploit int "shell.php" file which you created for uploading on the website.
- Submit the "filename" with which you uploaded the file in "filename.txt" file available in "/home/labDirectory"

Click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.

ACTIVITY-1: REMOTE CODE EXECUTION VIA SIMPLE ARBITRARY FILE UPLOAD

ACTIVITY-2: REMOTE CODE EXECUTION WITH BLACKLISTED EXTENSIONS BYPASS

ACTIVITY-3: REMOTE CODE EXECUTION WITH WHITELISTED FILE EXTENSION AND IMAGE CHECK BYPASS

DOWNLOAD ALL SUBMISSIONS

Activity-2: Remote Code Execution with Blacklisted Extensions Bypass

ATTEMPT DOWNLOAD SUBMISSION

Activity-2: Remote Code Execution with Blacklisted Extensions Bypass

Objective: Overcome a blacklist to upload files containing commands to obtain sensitive information

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

When you click on your name, the website will take you to a page "http://localhost:30000/userDetails.php" which lets you change your user details, like email and profile photo.

Ideally the update profile picture feature should allow you to only upload image files (i.e. jpeg,jpg,png), but this has a file upload vulnerability.

This lab is very similar to the earlier lab, except that the web-application has implemented blacklisting based defensive mechanism which will block all the file uploads with ['php', 'php2', 'php3', 'php4', 'php5', 'phtml', 'shtml'] extensions.

Your job is to overcome this defense and upload a malicious "PHP" script that prints the output of the "id" command. If you do this correctly, you will receive a flag.

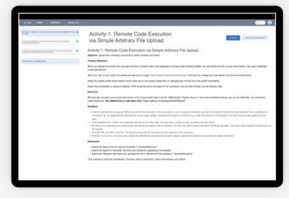
Guidance:

- First check how the defense works, so follow the same steps as in the earlier activity i.e. upload the shell.php and see what happens. You should get an error.
- Maybe .htaccess can help here? So determine the steps needed to make this work and execute them. Be sure to choose an extension for the php that is not listed above.
- Note when you try to upload files in the browser via the file handler of the OS, not all files may be visible. You will have to select to display all files and not just that, you need to display even hidden files (e.g. Ctrl+H will work in linux for hidden files).
- Once you successfully upload the php file, be sure to execute the script yourself and see the output of the id command. For this, you need to figure out where the file got uploaded. Use firefox page inspector to determine the file location.
- Visit that URL and click on the file. This should execute the file and show you the response of the execution.

Submission

- Submit the flag you find via "flag.txt" available in "/home/labDirectory".
- Submit the exploit int "shell.php" file which you created for uploading on the website.
- Submit the "filename" with which you uploaded the file in "filename.txt" file available in "/home/labDirectory"

Click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.



ACTIVITY-1: REMOTE CODE EXECUTION VIA SIMPLE ARBITRARY FILE UPLOAD

ACTIVITY-2: REMOTE CODE EXECUTION WITH BLACKLISTED EXTENSIONS BYPASS

ACTIVITY-3: REMOTE CODE EXECUTION WITH WHITELISTED FILE EXTENSION AND IMAGE CHECK BYPASS

DOWNLOAD ALL SUBMISSIONS

Activity-3: Remote Code Execution with Whitelisted File Extension and Image Check Bypass

ATTEMPT DOWNLOAD SUBMISSION

Activity-3: Remote Code Execution with Whitelisted File Extension and Image Check Bypass

Objective: Overcome a whitelist as well as Image check to upload files containing commands to obtain sensitive information

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

When you click on your name, the website will take you to a page "http://localhost:30000/userDetails.php" which lets you change your user details, like email and profile photo.

Ideally the update profile picture feature should allow you to only upload image files (i.e. jpeg,jpg,png,gif), but this has a file upload vulnerability.

This lab is very similar to the earlier lab, except that the web-application has implemented a whitelisting based defensive mechanism where it will only accept image files ending in jpg/jpeg/png/gif etc. Further, it also checks if the header of the files matches allowed MIME types. Remember image file headers have specific sequences of bytes at the beginning that identify the file format. JPEG files start with bytes FF D8 FF, while gif files start with the ASCII characters GIF87a or GIF89a.

Your job is to overcome this defense and upload a malicious "PHP" script that prints the output of the "id" command. If you do this correctly, you will receive a flag.

Note: There is no correlation between extensions

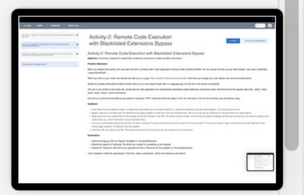
Guidance:

- First check how the defense works, so follow the same steps as in the earlier activity i.e. upload the shell.php and see what happens. You should get an error.
- Since it is allowing only files with specific extensions, try to name the file accordingly so that it will allow it to be uploaded (while at the same time php interpreter will also interpret it as a php file)
- You will notice that just renaming will not work, you also need to handle the header check. So, change the php file accordingly to handle this aspect as well. Gif files work better because the header is ascii.
- Once you successfully upload, be sure to execute the script yourself and see the output of the id command. For this, you need to figure out where the file got uploaded. Use firefox page inspector to determine the file location.
- Visit that URL and click on the file. This should execute the file and show you the response of the execution.

Submission

- Submit the flag you find via "flag.txt" available in "/home/labDirectory".
- Submit the exploit int "shell.php" file which you created for uploading on the website.
- Submit the "filename" with which you uploaded the file in "filename.txt" file available in "/home/labDirectory"

Click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.



ACTIVITY-1: SIMPLE PATH TRAVERSAL
ACTIVITY-2: PATH TRAVERSAL WITH SIMPLE DEFENSIVE CHECKS
DOWNLOAD ALL SUBMISSIONS

Activity-1: Simple Path Traversal

ATTEMPT DOWNLOAD SUBMISSION

Activity-1: Simple Path Traversal

Objective: Obtain sensitive information via path traversal with no safeguards

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "**Laksh:SecretPwd!"**

This website has a simple path traversal vulnerability. Leverage the vulnerability to read the "/etc/passwd" file and display it on the web page. Once the file is displayed, you will get a secret flag.

Guidance:

A few hints to help you out are as below:

- You should normally navigate through the entire website to find the vulnerable page, however to make things faster, the vulnerability is in the course details page. So, explore only this page
- Use firefox page inspector to explore this page
- To launch the vulnerability, you need to identify the right field in that page and edit it to the right value that launches the attack and relaunch the request. All this you can do via the page inspector tool itself.

Submission:

- Submit the payload you used to read the "/etc/passwd" file in exploit.txt
- Submit the flag in flag.txt available in "/home/labDirectory".

Click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.

ACTIVITY-1: SIMPLE PATH TRAVERSAL

ACTIVITY-2: PATH TRAVERSAL WITH SIMPLE DEFENSIVE CHECKS

DOWNLOAD ALL SUBMISSIONS

Activity-2: Path Traversal with simple defensive checks

ATTEMPT DOWNLOAD SUBMISSION

Activity-2: Path Traversal with simple defensive checks

Objective: Obtain sensitive information via path traversal with some simple safeguards

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

This website has a simple path traversal vulnerability. Leverage the vulnerability to read the "/etc/passwd" file and display it on the web page. Once the file is displayed, you will get a secret flag.

This is similar to the earlier activity, except that the page has some simple defenses in place. That is, if you tried the solution that worked in earlier activity here, it will block that URL and will not display the file. You need to overcome the defenses.

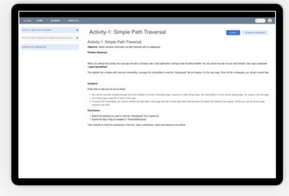
Guidance:

- There are two defenses in place, what they are and how to overcome, for you to figure out :-)

Submission:

- Submit the payload you used to read the "/etc/passwd" file in exploit.txt
- Submit the flag in flag.txt available in "/home/labDirectory".

Click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.



ACTIVITY-1: PASSWORD BRUTEFORCE WITH IP BLOCKING MECHANISM IN PLACE
ACTIVITY-2: USERNAME ENUMERATION WITH DEFENSE IN THE FORM OF ACCOUNT BLOCKING
ACTIVITY-3: BROKEN 2FA LOGIC AND BRUTEFORCING 2FA
ACTIVITY-4: BROKEN FORGOT PASSWORD LOGIC
DOWNLOAD ALL SUBMISSIONS

Activity-1: Password bruteforce with IP blocking mechanism in place

ATTEMPT DOWNLOAD SUBMISSION

Activity-1: Password bruteforce with IP blocking mechanism in place

Objective: Crack the admin password with a defense in the form of blocked IP after a few unsuccessful attempts

Problem Statement

When you attempt this activity, the cLab app will start a container with a web application running at http://websec-bodhi.duckdns.org:30000.

You do not need to login. Your job is to determine the password of a user with username admin. However the login mechanism has a defense where if you try multiple incorrect username-passwords, after 5 attempts, IP will be blocked by the application for sometime (few tens of minutes).

You dont have unlimited time, can you crack by bypassing the defense? Once you login as admin, you will get a flag.

Guidance:

- First check the defense i.e. try logging in with admin username and different passwords and see what happens. Post 5th attempt, IP should be blocked.
- If the IP is being blocked, a good technique to see is if each login attempt can use a different IP address? (Can change IP address after every 4 attempts also). It is also important to check at what layer IP address is being blocked, at network or application? Important to check since the attack vector will be a function of this. In this activity since we are dealing with web applications, let us assume that the blocking is happening at application level.
- Given above, would X-Forwarded-For header work? This header is a standard HTTP header used to identify the originating IP address of a client connecting to a web server through an HTTP proxy or load balancer. See reference. Normally this header is filled by proxies or load balancers, but clients can also add this header.
- What above means is, each request needs two fields fuzzed. One is a header i.e. X-Forwarded-For which takes on different IP addresses for each request and the other is a payload field i.e. field in the form which holds the password. You are given a "passwords.lst" file which contains a list of passwords. For IP addresses, you can take from a set say 10.10.*.* (this range should suffice).
- To fuzz two values in a request (disjointly) is a bit complicated. It is easy to do in Burpsuite, but no so in ZAP (needs custom scripts). Instead, we have provided a python template (exploit.py) as reference. You need to edit it and add code (fill in the blanks) as per requirement. (If you are finding this hard since you are not used to python, ask TA to share the exploit.py code).
- **NOTE: In the beginning of the exploit script there is a "DO NOT CHANGE" section. You must not change this section.**
- Post this, run the script (**python3 exploit.py** in the terminal) and determine the password. Login with the credentials to get the flag.
- In case you want to use ZAP and custom scripts (on your own), the container hosts zap at http://localhost:30004/zap/.

Submission:

- Submit the exploit.py available in "/home/labDirectory"
- Submit the flag in flag.txt available in "/home/labDirectory".

Reference:

https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Forwarded-For

ACTIVITY-1: PASSWORD BRUTEFORCE WITH IP BLOCKING MECHANISM IN PLACE

ACTIVITY-2: USERNAME ENUMERATION WITH DEFENSE IN THE FORM OF ACCOUNT BLOCKING

ACTIVITY-3: BROKEN 2FA LOGIC AND BRUTEFORCING 2FA

ACTIVITY-4: BROKEN FORGOT PASSWORD LOGIC

DOWNLOAD ALL SUBMISSIONS

Activity-2: Username Enumeration with defense in the form of account blocking

ATTEMPT DOWNLOAD SUBMISSION

Activity-2: Username Enumeration with defense in the form of account blocking

Objective: Enumerate usernames to obtain a valid username. Post this crack the password as well.

Problem Statement

When you attempt this activity, the cLab app will start a container with a web application running at http://websec-bodhi.duckdns.org:30000. You can login via "Laksh:SecretPwd!". ZAP is available at http://localhost:30004/zap/.

This task is different from activity 1. There you cracked the password of the user with username admin. Here, you dont know any valid username. Your first job is to enumerate usernames and find one valid username. Post this, your second job is to determine the password corresponding to the valid username. There is some defense here as well in the form of blocking, but the implementation logic is flawed. You need to figure out what the flaw is and accordingly launch the attack.

To simplify username enumeration, let us assume a valid user starts with "admin_" and then there are extra 2 random characters between [a-z] as suffix, all lower case. For example it could be "admin_vc". Note, in real life, you could use a file with random/popular names for this, but through this example, we are showcasing regular expression use in fuzzing in ZAP.

Once you login with a valid username and password, you will get a flag.

Guidance:

- To enumerate a valid username, you should first check what feedback the website provides against a valid and invalid username. You need to be a bit clever here and do something extra to see a difference.
- Once you determine what needs to be done for username enumeration, you can use ZAP to fuzz. In ZAP you will have to fuzz two fields in tandem. Unlike the kind of fuzzing needed in activity 1, this type of fuzzing in tandem is possible in ZAP.
- Select the relevant request (POST) in the main window and right click to select open/resend with request editor.
- The first field is obviously username. Select the relevant portion and then select fuzz and follow the steps like before. But here, you will have to select Regex (experimental). The regex to cover strings aa to zz is [a-z]{2}. Be sure to click generate preview to check the pattern.
- The second field can be a number, for this select numbers and fill the relevant fields accordingly.
- Look at response code or header or body size to identify differences in response to identify the right username.
- Post identifying the right username, a password list (passwords.lst) is provided to determine the right password. Again use ZAP to fuzz this.
- Will the defense block this like in activity-1? That is for you to figure out. Hint: IP blocking is not implemented in this lab :-)
- Based on this, think also as to how the password check is being implemented and what is the logical flaw there (if any)?

Submission:

- Submit the flag in flag.txt available in "/home/labDirectory".

ACTIVITY-1: PASSWORD BRUTEFORCE WITH IP BLOCKING MECHANISM IN PLACE
ACTIVITY-2: USERNAME ENUMERATION WITH DEFENSE IN THE FORM OF ACCOUNT BLOCKING
ACTIVITY-3: BROKEN 2FA LOGIC AND BRUTEFORCING 2FA
ACTIVITY-4: BROKEN FORGOT PASSWORD LOGIC
DOWNLOAD ALL SUBMISSIONS

Activity-3: Broken 2FA logic and Bruteforcing 2FA

ATTEMPT DOWNLOAD SUBMISSION

Activity-3: Broken 2FA logic and Bruteforcing 2FA

Objective: Login as admin without knowing the password in a system with 2FA

Problem Statement

When you attempt this activity, the cLab app will start a container with a web application running at http://websec-bodhi.duckdns.org:30000. You can login via a valid user account "Laksh:SecretPwd!"

Observer when you login, that there is a 2FA in place. For convenience, we have provided an email-Notification page where you can get the OTP (instead of sending it to your phone). **Please use open in new tab when you click on the email notification**. Note OTP will be valid for a few minutes, and it has between 0000 to 9999 possible values. Also, you get to see your OTP (i.e. Lakshs), not other peoples OTP through this page.

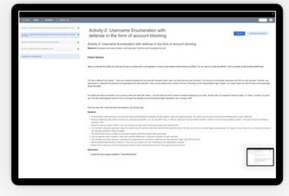
There is an implementation flaw in how the 2FA is implemented, your task is to carefully observe how users are being authenticated and then figure out a way to login as user admin without knowing the admin password or whatever OTP is sent to the admin.

Guidance:

- Use ZAP and capture all the requests being sent while doing authentication with Laksh username and OTP.
- Go through the requests being sent carefully to identify what is happening and zero down on requests that will let you do your task. Hint: You need to edit two requests.
- Select the first request, in the main window, right click and select open/resend with request editor. Edit it as needed and send.
- Then identify the second request. This request will involve fuzzing the OTP.
- In the main window, right click and select fuzz.
- Then edit the fields that needed to be modified but stay constant across fuzzing. Save these.
- Then select the otp and fuzz it. Here you will again need to use regular expressions for OTP format. It is \d{4}. Be sure to click generate preview to see if the expression is correct. Note, you also need to go till 9999, ensure this is happening as well.
- Look at the response code or header or body size to identify differences in response to identify the right otp.
- Post this, if you visit the web application, you should get a flag to submit. Note, you already sent the OTP via zap, so flag is generated.

Submission:

- Submit the flag in flag.txt available in "/home/labDirectory".



ACTIVITY-1: PASSWORD BRUTEFORCE WITH IP BLOCKING MECHANISM IN PLACE
ACTIVITY-2: USERNAME ENUMERATION WITH DEFENSE IN THE FORM OF ACCOUNT BLOCKING
ACTIVITY-3: BROKEN 2FA LOGIC AND BRUTEFORCING 2FA
ACTIVITY-4: BROKEN FORGOT PASSWORD LOGIC
DOWNLOAD ALL SUBMISSIONS

Activity-4: Broken Forgot Password Logic

ATTEMPT DOWNLOAD SUBMISSION

Activity-4: Broken Forgot Password Logic

Objective: Login as admin without knowing admins password by leveraging a broken forgot password implementation

Problem Statement

When you attempt this activity, the cLab app will start a container with a web application running at http://websec-bodhi.duckdns.org:30000. Laksh is a valid username.

The website has a reset forgotten password mechanism. Explore this mechanism and see if you can reset admin's password and login as admin. Once you login as admin, you will get a flag.

Guidance:

- There is no need for ZAP or for that matter firefox developer tools, just explore the mechanism and figure out a way. It is very simple!

Submission:

- Submit the vulnerable URL(s) that leads to changing admin's password in urls.txt
- Submit the flag in flag.txt available in "/home/labDirectory".

ACTIVITY-1: VERTICAL PRIVILEGE ESCALATION VIA URL BRUTE FORCING AND COOKIE MODIFICATION
ACTIVITY-2: HORIZONTAL PRIVILEGE ESCALATION VIA IDOR
ACTIVITY-3: ANOTHER PRIVILEGE ESCALATION BASED ON USER ROLE
DOWNLOAD ALL SUBMISSIONS

Activity-1: Vertical Privilege Escalation via URL brute forcing and cookie modification

ATTEMPT DOWNLOAD SUBMISSION

Objective: Access prohibited URLs and leverage them to alter web application state

NOTE: Make sure to delete all cookies on your browser before attempting the activity!

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://websec-bodhi.duckdns.org:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

You need to perform 2 tasks here.

Part-A: Access an admin panel

The developers essentially are doing security by obscurity by creating an admin URL that is not easily guessable, hoping only those who know the correct URL can access it. But attackers can overcome this via brute force attacks. Assume, the prefix to the admin panel is "http://websec-bodhi.duckdns.org:30000/v1/". Your task is to determine the correct admin URL and add a user with username "**Virat**" (who is not supposed to be part of the course :-)

Part-B: Access teachers page:

The website has a page with URL "http://websec-bodhi.duckdns.org:30000/teachers". This is well known, no security by obscurity here. Each user is assigned a role, when you login and that user has the role set to teacher, they can access this page, else no. Some access control is in place, however the implementation is flawed. Your job is to access this page, even though the user account you have access to (Lakshs) is not allowed this access. Once you access this page, change Lakshs currently poor end sem marks of course "CS-224: Computer Networks" to 100 :-)

Guidance:

PartA:

- One just needs to brute force and try a variety of popular strings. http://websec-bodhi.duckdns.org:30000/v1/xyz (where xyz is the string to try). The steps to follow are very similar to ZAP activity 3 fuzzing. There is a list of strings to try also provided in the lab folder.
- Be sure to login with Lakshs credentials before using ZAP. Web app expects some session id to deliver admin page, else will keep redirecting to login page.
- As part of ZAP fuzzing, once you identify a distinct URL whose response differs from others, access that URL and then add the required user.

PartB

- As has been hinted in the title, it appears that access control is cookie based. Examine the cookies under storage and see if altering them will give you access to the page?
- No need for ZAP here.

Once you add a user with username "Virat" and change the marks of "Laksh " for endsem of "CS-224 Computer Networks" course to 100, you will get a flag.

Submission:

- Submit the flag you find via "flag.txt" available in "/home/labDirectory".

ACTIVITY-1: VERTICAL PRIVILEGE ESCALATION VIA URL BRUTE FORCING AND COOKIE MODIFICATION

ACTIVITY-2: HORIZONTAL PRIVILEGE ESCALATION VIA IDOR

ACTIVITY-3: ANOTHER PRIVILEGE ESCALATION BASED ON USER ROLE

DOWNLOAD ALL SUBMISSIONS

Activity-2: Horizontal Privilege Escalation via IDOR

ATTEMPT DOWNLOAD SUBMISSION

Objective: Obtain sensitive information of other users via IDOR based privilege escalation

NOTE: Make sure to delete all cookies on your browser before attempting the activity!

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://websec-bodhi.duckdns.org:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

The website contains a help page accessible via "http://websec-bodhi.duckdns.org:30000/help/?userid=23208" or by clicking on "Help" on "user" page where a user can chat with an admin. This page however does not have proper access-control and it is possible for you to look at other student's chat-history. One of the chat histories has a students password. Login as this user to get a flag.

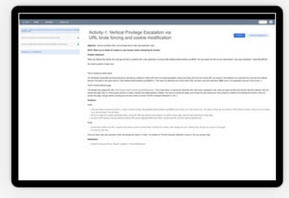
There is yet another IDOR vulnerability. Can you leverage it to extract the private phone number of user "Anita"?

Guidance:

- To determine password, look carefully at the URL of the help page and see how you can access other people's chat history. You can use ZAP here, but it is not necessary. You can just make a few guesses and try.
- For a phone number, you have to first identify the URL. Where do you think the phone number of a user is to be found? Post this URL identification, the task is more or less the same as the earlier task. However ZAP may be useful here, since the range to search is bigger (hint: +/- 100 numbers should do).
- In Zap, you will need to use numberzz (and not strings/file), this interface should be obvious to use.

Submission

Submit the flag you find as well as the phone number of Anita in "solution.txt" available in "/home/labDirectory".



ACTIVITY-1: VERTICAL PRIVILEGE ESCALATION VIA URL BRUTE FORCING AND COOKIE MODIFICATION

ACTIVITY-2: HORIZONTAL PRIVILEGE ESCALATION VIA IDOR

ACTIVITY-3: ANOTHER PRIVILEGE ESCALATION BASED ON USER ROLE

DOWNLOAD ALL SUBMISSIONS

Activity-3: Another privilege escalation based on User role

ATTEMPT DOWNLOAD SUBMISSION

Objective: Circumvent access control to modify end-user data

NOTE: Make sure to delete all cookies on your browser before attempting the activity!

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://websec-bodhi.duckdns.org:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

Laksh is a regular student. His access is more or less view only i..e he can see details of course, who his classmates are and his grades.

Rajesh is his friend who is a TA on the web application platform, and has some enhanced privileges. You can explore his access by logging in with credentials "Rajesh:Qwerty@123". As you can see, there is an admin panel and as part of this, he can upgrade grades (see top bar after clicking admin panel). But he can change grades of students only in courses he is a TA in.

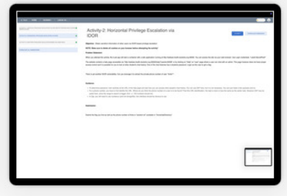
Now Laksh wants to change his grades in a course he is enrolled in CS224 (computer networks). Rajesh can change grades of students in courses of which he is TA, but he is not a TA in CS224 and hence cannot directly change Laksh;s grade.

So is there a way for Laksh or Rajesh to still change Lakshs grades? In this activity your task is to change Laksh's endsem marks of the course "CS224: Computer Networks" to 100.

Once you change the marks, you should get a flag.

Guidance:

- Login as Laksh and explore the website (including grades interface)
- Login as Rajesh and explore the website (specifically admin and update grades interface)
- Ensure foxyproxy is disabled, open ZAP, disable hud and then enbable foxy proxy so that the traffic of the website is reflected in ZAP. Specifically URLs associated with edit marks under admin panel (as part of Rajeshs exploration).
- Identify the URL that was used to update marks under Sites. Select the request button (next to quickstart). Then in the main request page, right click and select Open/resend with request editor. Your job is now to edit this request and send it again, so Lakshs marks are changed.
- **NOTE: There are 3 roles in the application: student, ta, admin**
- You can check if indeed marks are changed by logging in as Laksh and checking his grades under the CS224 course.
- All of above was done as Rajesh i.e. under Rajeshs session id, and since Rajesh was not TA of the course CS224, you would not have seen any changes, but, we are not done yet ...Can Laksh change the marks himself?
- For this, he needs to know the POST request details, which maybe Rajesh could share with him. And based on this Laksh could construct his own POST.
- But could he change the marks with Laksh;s session id? You can try this out also.
- In Zap, when you are editing the POST request, you can change any and all parameters and create an altogether new POST (has no bearing on original post). So, let us assume this POST is now being constructed by Laksh.
- Login as Laksh and explore either via ZAP or firefox developer tool, what is being sent in the requests. Specifically identify session state.
- Now alter the POST request wherein you use Lakshs session state and check if Laksh can still change marks? Hint: Apart from form data, you also need to change cookie data.
- **NOTE: There are 3 roles in the application: student, ta, admin**



Submission:

- Submit the flag you find via "flag.txt" available in "/home/labDirectory".
- Submit the "cookies" you used and the "POST request data" you used to change marks of Laksh in "postdata.txt" (Follow the instructions given in file for submission)

ACTIVITY-1: AUTHENTICATION BYPASS USING SQL INJECTION
ACTIVITY-2: DATA EXFILTRATION USING UNION BASED SQL INJECTION TECHNIQUE
ACTIVITY-3: DATA EXFILTRATION USING BLIND SQL INJECTION
DOWNLOAD ALL SUBMISSIONS

Activity-1: Authentication Bypass using SQL Injection

ATTEMPT DOWNLOAD SUBMISSION

Activity-1: Authentication Bypass using SQL Injection

Objective: Login as admin by exploiting a SQL injection vulnerability in the login page

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000.

You do not need to login. Leverage the login fields themselves to submit a payload that results in a SQL injection attack whereby you can login as admin even without knowing the password.

Guidance:

- Not much to say, use the login fields on the login page to launch the attack
- A few things to note:
- In SQL standard, the -- comment delimiter must be followed by a space to be recognized as a comment
- Now in web applications, when you add a space at the end, the protocol will trim it. So, you need to do something to handle this.

Submission:

- Once you login as "admin" and you will get a flag. Submit the flag in "/home/labDirectory/flag.txt".
- You also need to submit the exploit that lent you admin access i.e. what did you enter in both the fields of login. Submit these in the exploit.txt file available in "/home/labDirectory" folder
- And click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.

ACTIVITY-1: AUTHENTICATION BYPASS USING SQL INJECTION
ACTIVITY-2: DATA EXFILTRATION USING UNION BASED SQL INJECTION TECHNIQUE
ACTIVITY-3: DATA EXFILTRATION USING BLIND SQL INJECTION
DOWNLOAD ALL SUBMISSIONS

Activity-2: Data Exfiltration using UNION based SQL Injection technique

ATTEMPT DOWNLOAD SUBMISSION

Activity-2: Data Exfiltration using UNION based SQL Injection technique

Objective: Perform a Union-Based SQL Injection to exfiltrate the password of "admin"

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

There is a search field in the participants page, which is vulnerable to SQL injection. Leverage it and perform a Union-Based SQL Injection to exfiltrate the password of "admin" user from a different table. Post this, login as admin to retrieve the flag.

Guidance:

- There are two tables here. One is a userdetails table which contains detailed information about users enrolled in courses. The search field is essentially accessing this table to retrieve some information. There is also another table named Users, which contains username and password details.
- To launch union based attacks, you need to know the internals of the database i.e. what tables exist, how many columns in each table, what is the data type etc. And this information also attackers need to figure out on their own. And there are interesting ways to figure this also, see https://portswigger.net/web-security/sql-injection/union-attacks.
- However, to make things simpler, some useful information is as below.
- The SQL command run by the web application as a consequence of search is as given: "SELECT sno, username, courseid, waitlisted FROM userdetails WHERE courseid = '$courseid' AND username = '$search' AND waitlisted = 0 "
- sno: Integer, Primary Key
- username: string
- courseid: Integer
- waitlisted: Integer but 0/1 only
- The Users table has three columns:
- sno: Integer, Primary Key
- username: String
- password: String
- Your job is to construct the right UNION query to enter in the search field so that you can dump the Users table to access the admin password
- A few things to note:
- In SQL standard, the -- comment delimiter must be followed by a space to be recognized as a comment
- Now in web applications, when you add a space at the end, the protocol will trim it. So, you need to do something to handle this.
- Once you find the admin password, login to get the flag.

Submission

- Login as admin and you will get a flag. Submit the flag in "/home/labDirectory/flag.txt".
- You also need to submit the exploit (i.e. what you typed in the search box) that revealed the admin password. Submit the exploit in exploit.txt file available in "/home/labDirectory" folder

And click evaluate to verify the submissions. Post this, make a submission, select local directory and submit.

ACTIVITY-1: AUTHENTICATION BYPASS USING SQL INJECTION
ACTIVITY-2: DATA EXFILTRATION USING UNION BASED SQL INJECTION TECHNIQUE
ACTIVITY-3: DATA EXFILTRATION USING BLIND SQL INJECTION
DOWNLOAD ALL SUBMISSIONS

Activity-3: Data Exfiltration using Blind SQL Injection

ATTEMPT DOWNLOAD SUBMISSION

Activity-3: Data Exfiltration using Blind SQL Injection

Objective: Perform a blind SQL Injection to exfiltrate the password of "admin"

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

Unlike earlier activities, this activity has a blind SQL injection. The attacker can execute some SQL command of his choice but the results are not directly visible to the attacker. This makes it more difficult to exploit and detect due to lack of feedback.

Once you login, enter any course. Then click on **participants**. Notice the URL of the loaded page , it is of the type http://localhost:30000/participants.php?courseid=4. A "courseid" is being used as a query parameter. Is this vulnerable to SQL injection?

Exploit this parameter to obtain the password of the admin user.

Guidance:

(This exercise is hard. You can use an automated tool like SQLmap to exploit the vulnerability for you. We will cover this further below, but the first few steps until bruteforcing do try out for better understanding of what is happening. Sqlmap also employs similar techniques to crack)

Testing the ground:

Lets test the ground to see if there is any sql injection possible.

- Lets try http://localhost:30000/participants.php?courseid=4
- Notice that the no of enrolled students shown is 12
- Lets now try another sql command: http://localhost:30000/participants.php?courseid=4%27%20and%201=1--%20abc (equivalent to http://localhost:30000/participants.php?courseid=4' and 1=1-- abc)
- This should give more than 12 students. Here it will give 17 since internally there is a waitlist column. The comment will comment out the waitlist parameter of the SQL query. Every row in the table is checked to see if courseid is 4 and 1=1 (which is always true), hence this will return all rows in the table where courseid=5 including participants that are waitlisted.
- Lets try a false statement now, http://localhost:30000/participants.php?courseid=4%27%20and%201=2--%20abc (equivalent to http://localhost:30000/participants.php?courseid=4' and 1=2-- abc)
- Notice that the no of enrolled students has changed, it is now 0. Since when checking the rows, 1=2 is false, so no row will be selected. Hence 0 rows.

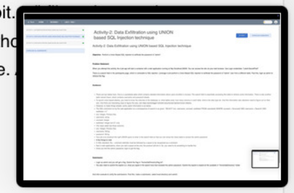
Since the result is varying based on injected condition, it is vulnerable to **blind** sql injection.

Now how to launch the attack? You should first determine the password length first and then the actual password.

You can assume the following:

1. There is a table called users which contains admin password
2. This table has 3 columns: `sno`, `username`, `password`
3. Possible characters in password: [a-zA-Z0-9_] (regex form) and max length is 50

Since this activity requires an exhaustive bruteforce, we have also given you an "exploit.py" file with skeleton code available in it. You may use it to automate your exploitation. Remember that the "exploit. necessary code to connect and send the payload, you will need to add/update payload as well as the authentication details in it. You can also change the logic of the code if you are comfortable with pytho have to specify a PHPSESSID here in the python script. This is because the page we are accessing is an authenticated page, the server will process the request only if you send the right session cookie. cookie is PHPSESSID. You can obtain this via storage option from firefox developer tools.



You can skip the brute forcing if you find it difficult, instead make use of sqlmap. Details below.

Helpful Tool: SQLMap

While you can do the exploitation manually (and you should since this is more or less what SQL Map is internally doing as well), you can also automate this via the command line tool SQLMap which is provided as part of the

ACTIVITY-1: AUTHENTICATION BYPASS USING SQL INJECTION
ACTIVITY-2: DATA EXFILTRATION USING UNION BASED SQL INJECTION TECHNIQUE
ACTIVITY-3: DATA EXFILTRATION USING BLIND SQL INJECTION
DOWNLOAD ALL SUBMISSIONS

You can assume the following:

1. There is a table called users which contains admin password
2. This table has 3 columns: `sno`, `username`, `password`
3. Possible characters in password: [a-zA-Z0-9_] (regex form) and max length is 50

Since this activity requires an exhaustive bruteforce, we have also given you an "exploit.py" file with skeleton code available in it. You may use it to automate your exploitation. Remember that the "exploit.py" file only contains necessary code to connect and send the payload, you will need to add/update payload as well as the authentication details in it. You can also change the logic of the code if you are comfortable with python programming. Note you have to specify a PHPSESSID here in the python script. This is because the page we are accessing is an authenticated page, the server will process the request only if you send the right session cookie. And the name of that cookie is PHPSESSID. You can obtain this via storage option from firefox developer tools.

You can skip the brute forcing if you find it difficult, instead make use of sqlmap. Details below.

Helpful Tool: SQLMap

While you can do the exploitation manually (and you should since this is more or less what SQL Map is internally doing as well), you can also automate this via the command line tool SQLMap which is provided as part of the container. Please find a detailed guide for SQLMap tool here: https://www.stationx.net/sqlmap-cheat-sheet/

SQLMap is quite a powerful tool, it can do many things. To try a vulnerability scan and ask it to dump all tables it found out, try

sqlmap -u 'http://localhost:30000/participants.php?courseid=5' --cookie='PHPSESSID=jh3c0eqqu03mlcvjh1ddjj1spr' --dump batch

-u : url to try.

For cookie: note since the page we are accessing is authenticated, you have to pass on the authentication cookie to sqlmap for it to access the page. This you can find out via storage option in firefox developer tools. You need only the PHPSESSID cookie. Be sure to change the cookie value to that you found.

dump: instructs sqlmap to extract (dump) the contents of the database tables it finds.

batch: runs sqlmap in batch mode, which means it will not prompt the user for any interactive input

This command may take many minutes. Once it exits, you should see the Users table with the admin password.

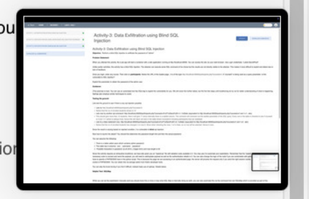
A faster command (which will still take 5-6 minutes still) is below.

sqlmap -u 'http://localhost:30000/participants.php?courseid=5' --cookie='PHPSESSID=v5t7jifqq5u6o5r7ndvce9an4f' --search -T users --batch

Here

--search: Tells sqlmap to search for specific items in the database.

-T users: Specifies the table name to search for within the databases (note, you normally wouldnt know the name of the table, but we had mentioned it earlier as hint. If you dont know the name, you shou
command, which does a full scan.)



Submission Details:

- Login as admin and you will get a flag, submit it in "/home/labDirectory/flag.txt". It will serve as proof of concept of your work. And click evaluate to verify the submissions. Post this, make a submissio
submit.
- You are also supposed to submit the exact "sqlmap" comand you used in "command.txt" OR If you have written a python script for the work then it needs to go in "exploit.py" file.

ACTIVITY-1: INTRODUCTION TO STORED AND REFLECTED XSS
ACTIVITY-2: REFLECTED XSS WITH ACTIVE BLACKLIST FILTRATION
DOWNLOAD ALL SUBMISSIONS

Activity-1: Introduction to stored and reflected XSS

ATTEMPT DOWNLOAD SUBMISSION

Activity-1: Introduction to stored and reflected XSS

Objective: Understand stored and reflected XSS and the differences therein

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

In XSS attacks, there are two roles one has to play. One is that of the attacker launching the attack and the other is that of the victim (end-user who is accessing the victim website and is affected, by say having his/her cookies stolen!)

You will assume the role of the attacker here and explore the website and do the homework to determine how to launch the attack and what payload to use to launch the attack. Post this, we have provided an exploit-server. The attacker (i.e. you) can specify via the exploit server what URL victim (browser) should access. And based on this, internally, we will simulate a victim (an administrator no less :-) who will then access the URL and get attacked. We will also check if indeed the victim was attacked the way intended! The exploit server is available "http://localhost:30000/exploit-server"

PART-1 (Reflected XSS):

Enter any course and then participants. This page has a search feature. Exploit this search feature to launch an XSS attack, where an alert box with the words "hacked" will pop for the victim when he/she visits the URL. Your task is to construct an appropriate URL and specify it in the exploit server. Post submit, you should get a flag.

PART-2 (Stored XSS):

Enter any course and then grades. This page has a comment feature, which lets you comment on your grades. Exploit this comment feature to launch an XSS attack, where an alert box with the words "hacked" will pop for the victim when he/she visits the URL. Your task is to construct an appropriate URL and specify it in the exploit server.

PART-3 (Stealing Cookie):

Your goal is to use any of the XSS you identified above to steal "admin" user's cookie. In fact you should try to do this with both, though you will get the flag once one of them succeeds. Your task is to construct an appropriate URL and specify it in the exploit server. Note, exploit server mimics admin action i.e. admin will visit the URL you constructed. However when the admin visits the URL and the cookie gets stolen, it has to be sent somewhere i.e. to some server under the attacker's control. We have also provided you with such a server. In your container terminal use the following command: "python3 -m http.server 30001". This will launch a server which listens on 30001 port and displays whatever it receives.

Your job is to construct the URL you type in the exploit server in such a way that not only will it steal the cookie but also send it to your server (running on localhost) listening on the specified port.The stolen cookie will contain the third flag for you to submit.

Guidance:

- **Part-1 and Part-2 are rather straightforward. Experiment with the payload against yourself first. Once you see the alert yourself, then copy the relevant URL to the exploit server and submit it to get flags.**
- Note that javascript code you inject may not be visible on the webpage. This is because it is being interpreted as javascript. You can use page inspector to see that indeed it got embedded in html.
- **Reg part-3, your javascript code instead of popping an alert has to steal the cookie(s) of whoever visits the page. And further pass it to some attacker server.**
- Try using the payloads available in this medium writeup: https://pswalia2u.medium.com/exploiting-xss-stealing-cookies-csrf-2325ec03136e
- **So, first, open a pop up terminal and be sure to type python3 -m http.server 30001 to start the server listening on port 30001 to receive the cookie.**
- Now on your own page, craft the payload so that once page loads and javascript executes, you should see your own cookie sent to the server (server will display whatever it receives). You can use a page inspector to see how the javascript is embedded.
- **Then use the right URL in the exploit server and click submit so that the admins cookie gets sent to the http server. This should reveal the flag.**

Submission:

- Each part you do above will have a separate flag, submit those flags in "/home/labDirectory/flag.txt" in new lines.
- Also submit the exploits in "/home/labDirectory/part1.txt", "/home/labDirectory/part2.txt", and "/home/labDirectory/pat3.txt" accordingly.

ACTIVITY-1: INTRODUCTION TO STORED AND REFLECTED XSS
ACTIVITY-2: REFLECTED XSS WITH ACTIVE BLACKLIST FILTRATION
DOWNLOAD ALL SUBMISSIONS

Activity-2: Reflected XSS with active blacklist filtration

ATTEMPT DOWNLOAD SUBMISSION

Activity-2: Reflected XSS with active blacklist filtration

Objective: Overcome a blacklist based defence to launch reflected XSS attack.

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

This activity is very similar to Part-1 of earlier activity. Enter any course and then participants. This page has a search feature. Exploit this search feature to launch an XSS attack, where an alert box with the words "hacked" will pop for the victim when he/she visits the URL. Your task is to construct an appropriate URL and specify it in the exploit server. Post submit, you should get a flag.

However the web application filters blacklisted words and characters. Words removed by application: script, alert. Characters converted to HTML encoding by application: <, >, "

Guidance:

- First, understand the defense i.e. proceed with the exploit as in part-1 of earlier activity. Carefully observe what is being printed to determine what the server is doing. You will notice that some words are being deleted, while some characters are html encoded.
- You somehow have to ensure that in spite of deleting, the relevant words appear, and somehow ensure the server does not see special characters so that they are not encoded.

Submission:

Once you get the flag, submit it via "/home/labDirectory/flag.txt"

- Also submit the exploit via "/home/labDirectory/exploit.txt"

ACTIVITY-1: SIMPLE CSRF WITH NO DEFENSIVE MECHANISM
ACTIVITY-2: CSRF ATTACK WITH IMPROPER CSRF TOKEN IMPLEMENTATION (TOKEN NOT TIED TO USER-SESSION)
ACTIVITY-3: CSRF ATTACK WITH IMPROPER CSRF TOKEN IMPLEMENTATION (CSRF TOKEN IS PAIRED WITH A COOKIE)
DOWNLOAD ALL SUBMISSIONS

Activity-1: Simple CSRF with no defensive mechanism

ATTEMPT DOWNLOAD SUBMISSION

Activity-1: Simple CSRF with no defensive mechanism

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!"

In CSRF attacks, there is an attacker website where the attacker hosts a payload which when clicked by the victim user when browsing will cause some damage to the victim user wrt some genuine website (referred to as victim website) in which the victim has an account.

We have provided an exploit server here as well. The role of the exploit server here is to mimic a victim visiting the attacker website. The payload that launches the CSRF attack needs to be specified in the exploit server (under body, rest fields you can ignore). The payload can be specified in the form of only relevant html/javascript code (no need to enter other code/tags like <head> , <body> etc, that we will take care). Based on the entered payload, we will construct an attacker web page with the payload and simulate a victim (an administrator no less :-) who will then visit the page and get attacked. We will also check if indeed the victim was attacked the way intended! The exploit server is available http://localhost:30001/

Now, what should the CSRF attack target? In this activity, it is essentially changing the victims email address on the victim website to that of the attackers, in the hope that the attacker can then potentially take over the victims account on the victim website, since forgotten password tokens, links etc are managed via emails. The victim website is http://localhost:30000. And the relevant page inside is http://localhost:30000/userDetails/, accessible when you click on username on the top bar. There is an update email field, where one can change their email address. Explore this page, including seeing what requests are being sent, what are the headers included etc (via firefox developer tools). Based on this exploration, construct a CSRF payload which when hosted on the exploit server, will change the admins email address to whatever you like. Remember we are simulating an admin visiting the attacker website via the exploit server, you can assume admin is already logged in the victim website.

Note there are no defenses in the form of CSRF token present in this exercise.

Guidance:

- In the exploit server, deliver to victim is mainly simulating admin accessing the attacker website, while view exploit will cause your own browser to visit the page (you are launching the attack against yourself). Try the attack first against yourself to verify correctness (check if your email has been updated).
- To find out what payload to use, you need to explore the relevant page well (http://localhost:30000/userDetails/). See what requests are being sent, what are the headers included etc (via firefox developer tools). Since we are using the same website (localhost:30000) for many labs, cookies/token across labs will tend to accumulate. For this lab, it would be a good idea to delete all cookies before you start exploration. You can do this by accessing storage on firefox developer tools, select cookies, then the relevant web site and right click to delete all.
- To construct a CSRF payload, you need to identify what is the URL end point to target and what are the fields you need to pass on to achieve the end goal. Basically boils down to creating a form and filling it with relevant fields and autosubmitting the form via javascript.
- Note: To view the flag, you have to move back from the exploit server to the main web application hosted at localhost:30000/ (open it in another tab).

Submission

Once your payload successfully changes the email of "admin" user, the web-application will give you a flag, submit it via "/home/labDirectory/flag.txt".

- Also submit your exact exploit in "/home/labDirectory/exploit.txt"

ACTIVITY-1: SIMPLE CSRF WITH NO DEFENSIVE MECHANISM

ACTIVITY-2: CSRF ATTACK WITH IMPROPER CSRF TOKEN IMPLEMENTATION (TOKEN NOT TIED TO USER-SESSION)

ACTIVITY-3: CSRF ATTACK WITH IMPROPER CSRF TOKEN IMPLEMENTATION (CSRF TOKEN IS PAIRED WITH A COOKIE)

DOWNLOAD ALL SUBMISSIONS

Activity-2: CSRF Attack with improper CSRF token implementation (token not tied to user-session)

ATTEMPT DOWNLOAD SUBMISSION

Activity-2: CSRF Attack with improper CSRF token implementation (token not tied to user-session)

Objective: Overcome an improper CSRF token implementation to launch a CSRF attack

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!" There is also an exploit-Server (to mimic admin user visiting attacker website) hosted at http://localhost:30001

This activity is very similar to the first activity with the same goal: change the admin's email address to one controlled by the attacker. However there is a defense in the form of a CSRF token here, so the same steps as before will not work. You should however try and confirm this. It should say no CSRF token found.

However there is an implementation flaw (how do you know? in practice this is not known, you just have to try different things and try to break). The flaw here is that the token is not tied to the user session i.e. attacker could get a CSRF token for self and use it for others.

Your task is to create a payload which will change the "email" of "admin" user and submit the payload in the exploit-server.

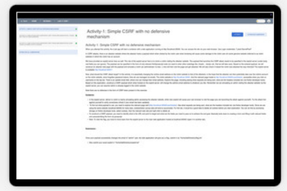
Guidance:

- Since we are using the same website (localhost:30000) for many labs, cookies/token across labs will tend to accumulate. For this lab, it would be a good idea to delete all cookies before you start exploration. You can do this by accessing storage on firefox developer tools, select cookies, then the relevant web site and right click to delete all.
- To find out what payload to use, you need to explore the relevant page well (http://localhost:30000/userDetails/). See what requests are being sent, what headers included etc via firefox developer tools. Look at both page inspector to find where CSRF token is hidden in the form as well as the network tab to see what all is in the POST, where is the CSRF being conveyed etc
- Once you understand this, construct an appropriate CSRF payload and submit it in the exploit server. Remember the flaw implies, you need to get a valid CSRF token first and then incorporate it in the payload.
- First check against yourself (view exploit) and then deliver to the victim.

Submission

Once your payload successfully changes the email of "admin" user, the web-application will give you a flag, submit it via "/home/labDirectory/flag.txt".

- Also submit your exact exploit in "/home/labDirectory/exploit.txt"



ACTIVITY-1: SIMPLE CSRF WITH NO DEFENSIVE MECHANISM

ACTIVITY-2: CSRF ATTACK WITH IMPROPER CSRF TOKEN IMPLEMENTATION (TOKEN NOT TIED TO USER-SESSION)

ACTIVITY-3: CSRF ATTACK WITH IMPROPER CSRF TOKEN IMPLEMENTATION (CSRF TOKEN IS PAIRED WITH A COOKIE)

DOWNLOAD ALL SUBMISSIONS

Activity-3: CSRF Attack with improper CSRF token implementation (CSRF token is paired with a cookie)

ATTEMPT DOWNLOAD SUBMISSION

Activity-3: CSRF Attack with improper CSRF token implementation (CSRF token is paired with a cookie)

Objective: Overcome an improper CSRF token implementation to launch a CSRF attack

Problem Statement:

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. You can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!" There is also an exploit-Server (to mimic admin user visiting attacker website) hosted at http://localhost:30001

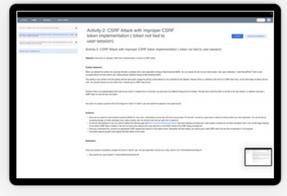
This activity is very similar to the earlier activity with the same goal: change the admin's email address to one controlled by the attacker.

However there is a defense in the form of a CSRF token here as well, with a slightly different implementation than in earlier activity. The web application, when it sends a form, installs a cookie and also embeds a CSRF token in the form. They both form a pair (they could be different in value, but we just made them have the same value in this activity for simplicity). When the user fills the form and sends it back, the server just needs to check if the cookie matches with the CSRF token returned with the filled form. If so, it will act on it, else reject the request. (Why implemented like this? this makes it stateless and hence simpler).

Given this implementation, your job is to construct a CSRF payload that will overcome this defense and change the "email" of "admin" user. Submit the payload in the exploit-server.

Guidance:

- Since we are using the same website (localhost:30000) for many labs, cookies/token across labs will tend to accumulate. For this lab, it would be a good idea to delete all cookies before you start exploration. You can do this by accessing storage on firefox developer tools, select cookies, then the relevant web site and right click to delete all.
- To find out what payload to use, you need to explore the relevant page well (http://localhost:30000/userDetails/). See what requests are being sent, what headers included, what cookies being sent, is the value of cookie same as the CSRF token etc via firefox developer tools. Look at both page inspector to find where CSRF token is hidden in the form as well as the network tab to see what all is in the POST, where is the CSRF being conveyed etc
- The big challenge in this lab is that the browser sends the cookie and the attacker cannot control the cookie. While the attacker can choose a value for the CSRF token, how can he ensure the same token is in the cookie as well?
- For this, you need to check if attacker can inject a cookie. For this, look at http://localhost:30000/participants/?courseid=2 and the search functionality within
- Explore this search page and see if the search term is influencing any cookie setting. If so, you have to inject a cookie with a value of your choosing via search.
- If you can do this, then you can bypass the implementation and launch a CSRF attack.
- Once you understand all of the above, think carefully and construct an appropriate CSRF payload and submit it to the exploit server. A bit hard, but doable.
- First check against yourself (view exploit) and then deliver to the victim.



Submission

Once your payload successfully changes the email of "admin" user, the web-application will give you a flag, submit it via "/home/labDirectory/flag.txt".

- Also submit your exact exploit in "/home/labDirectory/exploit.txt"

ACTIVITY-1: BASIC ORIGIN REFLECTION
ACTIVITY-2: TRUSTED SUBDOMAINS
DOWNLOAD ALL SUBMISSIONS

Activity-1: Basic Origin Reflection

ATTEMPT DOWNLOAD SUBMISSION

Activity-1: Basic Origin Reflection

Objective: Exploit flawed implementation of CORS to retrieve sensitive information

Credentials for testing: "Laksh:SecretPwd!"

Problem Statement:

In attacks involving CORS, there are three entities. One is the attacker website and another is victim website, and the last is ofcourse the end-user (browser) which facilitates the attack by clicking something in the attackers website. Wrt end-user, there is you as an end-user who is donning the hat of the attacker to figure out how to launch the attack by interacting with the victim website to figure out payload to use on the attacker website. But the main end-user is an admin (victim end-user) who will click something on the attacker website (the payload you launched).

When you attempt this activity, the cLab app will start a container with a web application running at http://localhost:30000. This is the victim website. You (attacker end-user) can access the site via your web browser. Use Login credentials: "Laksh:SecretPwd!". To mimic the admin end-user visiting an attacker website, we also provide an exploit-Server hosted at http://localhost:30001, where you need to specify the payload. The deliver to victim button mimics the admin user visiting the attacker website containing the payload.

The victim website has an insecure CORS configuration in that it trusts all origins and just reflects the same origin in the response. Look at guidance that explains the setup in more detail. To solve the lab, craft JavaScript code that uses CORS to contact the right api endpoint to get some sensitive information and then dumps the response (which contains admin userid etc) to the attacker website console (which is nothing but the exploit-server console), whose logs you can access via "http://localhost:30001/log". The lab is solved when you successfully log the response to the log file, you will get a flag.

Guidance:

- Your first task is to explore how the user-id is being fetched. Use firefox developer tools (network tab for this). For each request, check the response as well (you can also select raw to see the raw html code). You will notice that there is a specific call being made to a specific api-end point to get user id.
- Once you identify the end-point, you need to check its CORS capability. Since you are within the same domain as the endpoint, browsers will not add origin header. You have to mimic what happens if this request was launched from another origin (i.e. attacker website). So, for this, you can edit the header being sent.
- Select the specific request which is fetching the userid
- Right click and select edit and resend
- Here add a new header called origin and in value, type any random URL (different origin from localhost:30000)
- See the request and response header for this resend request. Use raw for better clarity. Notice the use of CORS fields. This should tell you what vulnerability the end-point has.
- Based on the above, craft an exploit involving javascript to contact this end point and then dump the resulting response to the log endpoint of the exploit server (something along the lines of location='/log?key='+this.responseText;). Be sure to include credentials in the request.
- In the exploit server, you are specifying a payload. Post this, we will mimic an admin user accessing this payload.
- The results of the payload should be such that the admin user will contact the vulnerable end-point with admin credentials (cookies etc) and once response comes, the payload will read the response and dump it to log endpoint of the exploit server.
- If you do this successfully, you should see a flag in the log.
- Note: if CORS were configured properly, when the admin user contacts the end point, the resulting response will not be allowed to be accessed by the attacker website code. Because of the vulnerability you are able to access.

Submission:

Submit the flag you found in the logs via "/home/labDirectory/flag.txt". Also submit your exact exploit in "/home/labDirectory/exploit.txt"

ACTIVITY-1: BASIC ORIGIN REFLECTION
ACTIVITY-2: TRUSTED SUBDOMAINS
DOWNLOAD ALL SUBMISSIONS

Activity-2: Trusted Subdomains

ATTEMPT DOWNLOAD SUBMISSION

Activity-2: Trusted subdomains

Objective: Exploit flawed implementation of CORS to retrieve sensitive information

Credentials for testing: "Laksh:SecretPwd!"

Problem Statement:

This activity is similar in spirit to earlier activity but with some significant changes. This lab is a bit complicated, go through guidance carefully before attempting this lab.

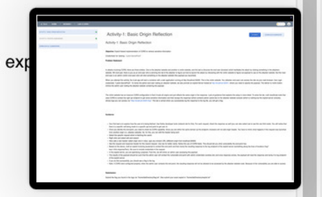
The victim website is "app.bodhitree.com:30000" (it was localhost:30000 earlier), but the behavior is same as before i.e. one of the endpoints here has CORS enabled and will pass on some sensitive info (userid). However, unlike earlier activity, the end point has a better CORS implementation , where it maintains a whitelist which allows only subdomains of bodhitree.com under access-control-allow-origin.

Given above, an attacker cannot easily launch the attack, the request to the CORS enabled endpoint has to come from some valid subdomain. This implies that some page within this website or hosted on related subdomains has to have some other vulnerability , XSS for example. In this case, you can inject malicious javascript into that page via XSS and then that javascript can trigger a request to the victim CORS enabled endpoint (this request will originate from whitelisted subdomain) and the response will be made available by browser to the malicious javascript which can then send it to some attacker endpoint.

To solve the lab, first identify the XSS endpoint which is vulnerable. Based on this, craft JavaScript code that uses CORS to contact the right api endpoint to get some sensitive information and then dumps the response (which contains admin userid etc) to the attacker website console (which is nothing but the exploit-server console), whose logs you can access via "http://localhost:30001/log". The lab is solved when you successfully log the response to the log file, you will then get a flag.

Guidance:

- In order to enable whitelist of sub-domains, we had to use some valid domain, and hence *.bodhitree.com. However the website corresponding to these domains are still running locally. So, we have to map these domains to local IP. For this,
- If in linux, edit "/etc/hosts" file as a root user (sudo) and add the line 127.0.0.1 app.bodhitree.com course.bodhitree.com at the top.
- If in windows, edit as administrator file C:\Windows\System32\drivers\etc\hosts and add the same line as above.
- Post this explore how the user-id is being fetched. This is similar to the earlier activity. Identify the specific api-end point to get user id.
- Check its CORS capability. Things are a bit convoluted here due to the way browsers work here.
- Much like in activity 1, select the specific request which is fetching the userid, right click and select edit and resend. Here add a new header called origin and in value
- First type a random URL (say http://www.example.com) and send. You will not see any CORS headers in reply. Maybe because the referrer field is still in the same domain.
- Type value as http://app.bodhitree.com:30000, this should invoke the right CORS response and you should see the headers. This shows CORS is implemented.
- Now determine where an XSS vulnerability exists and check if it works. Hint: It is reflected XSS.
- Based on above, craft an exploit involving javascript to first inject XSS and as part of the injection, contact the vulnerable end point and then dump the resulting response to the log endpoint of the exp along the lines of location='/log?key='+this.responseText;). Be sure to include credentials in the request.
- Note view exploit may throw an error, but that doesnt mean exploit did not work, check via access logs to see if logs capture what is intended.
- Post this, deliver to victim, if you do this successfully, you should see a flag in the access logs



Submission: